Ders 3 Genişletilmiş Notu: UML Diyagramları ve Nesneye Yönelik Analiz

Ders 3 Genişletilmiş Notu: UML Diyagramları ve Nesneye Yönelik Analiz

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Mimari ve tasarım nasıl ifade edilir?
- 2. Diyagramların kısa tarihi: akış diyagramından OMT'ye
- 3. UML'in doğuşu ve analiz-tasarım ayrımı
- 4. Yapısal ve davranışsal diyagramlar; diyagram kullanmanın faydası ve zararı
- 5. Yapısal diyagramlar
- 6. Davranışsal diyagramlar
- 7. Nesne Kısıt Dili (OCL) ve UML meta modeli
- 8. Nesneye yönelik analiz: neden nesneler?
- 9. Dilbilgisinden modele: ağaç yapraklarını sayma örneği
- 10. UML sınıf modelinin ayrıntıları
- 11. İleri ilişkiler: ilişki sınıfları, birleşme, bütünleşme, niteleyiciler
- 12. Genelleme ve kısıtları
- Örnek Sorular

Ders iki saatten oluşur. İlk saatte UML'in 14 diyagramı tek tek tanıtılır ve bir mimariyi ifade ederken hangilerinin işe yarayacağı tartışılır (slayt destesi BLM5126-03, 47 slayt). İkinci saatte nesneye yönelik analizin nasıl yapıldığı bir örnekle gösterilir, sonra UML sınıf modelinin ayrıntılarına geçilir (slayt destesi BLM5126-04, 43 slayt) (ekrandan, 00:10:00 ve 01:20:00).

1. Mimari ve tasarım nasıl ifade edilir?

Hoca dersin sorusunu şöyle koyar: bir yazılımın mimarisi, yani mimari tasarımı nasıl ifade edilir? Bunu hangi dilde, hangi araçlarla görselleştirebiliriz? Bu derste kullanılacak temel dil UML'dir. Hoca, öğrencilerin UML'in bazı diyagramlarını iyi bildiğini, bazılarının ise adını bile duymamış olabileceğini düşünür. Bu yüzden bütün diyagramlar tanıtılacak, her birinin bir yazılım mimarisini modellerken ne işe yaradığı gösterilecektir. Bir mimariyi gösterirken bütün diyagramları kullanmak gerekmez; diyagramlar tanındıkça hangilerinin işe yarayacağı da ortaya çıkar.

2. Diyagramların kısa tarihi: akış diyagramından OMT'ye

Diyagramlar yalnız UML'e özgü değildir; yazılım geliştirmede en başından beri popülerdir. Algoritmalar akış diyagramlarıyla ifade edilir ve akış diyagramlarının belirli sembolleri vardır. Hoca, eskiden algoritmaları kâğıda çizmek için kullanılan, sembol kalıpları oyulmuş bir cetveli kameraya gösterir; kendisinde de böyle bir cetvel vardır.

Zaman içinde yazılım dünyasında pek çok karmaşık diyagram tekniği ortaya atıldı: Jackson Design, Structured Design ve benzerleri. Hoca bunlara girmez; adlarından yola çıkılarak araştırılabileceğini söyler.

80’lerin sonundan itibaren ve 90’larda nesneye yönelik teknikler yaygınlaştı ve diyagramlar da değişmeye başladı. Bunların en önemlisi OMT’dir. OMT, o yıllarda General Electric’te çalışan James Rumbaugh ve ekibi tarafından ortaya atıldı. Ekip tekniği bir kitapla da yayımladı: *Object-Oriented Modeling and Design with UML* (hocanın gösterdiği, ikinci baskının kapağıdır).

> **Ders sonrası not:** Hoca OMT’yi “Object Management Technique” diye açtı; doğrusu Object Modeling Technique’tir. “Object Management” adı, UML’i daha sonra standartlaştıran Object Management Group’a (OMG) aittir.

Kitabın kapağında bir küp vardır ve küpün görünen üç yüzünde üç farklı diyagram çizilidir. Sol alt yüzde bir sınıf diyagramı, sağ alt yüzde bir durum geçiş diyagramı (state chart), üst yüzde bir ardışıl diyagram vardır. Bu, çeşit çeşit diyagram sunulduğunu değil, üç bakış açısının bir uyum içinde birleştirildiğini simgeler. Bir yazılımın yapısal, davranışsal ve işlevsel yönleri bir arada ifade edilebilmelidir. OMT’nin rakipleri de gelişiyordu, ama üç görünümü ortak bir paydada, bir diyagram ailesiyle göstermesi sayesinde OMT öne çıktı.

3. UML’in doğuşu ve analiz-tasarım ayrımı

3.1 OMT’den UML’e

Slayttaki özet şöyledir (ekrandan, 00:10:00):

- OMT, değişik teknikleri bir araya toplama çalışmalarının gelişmesine neden oldu. En bilineni Tümleştirilmiş Modelleme Dili olan UML’dir (Unified Modeling Language).
- OMG (Object Management Group) tarafından standart hâline getirildi. Teknik şartnameler OMG’nin sayfasında bulunabilir.
- IBM tarafından da desteklendi: Rational firması, Rational Rose ürünü.
- Pek çok ticari ve bedava ürün mevcuttur.
- UML’in üç ana mimarı vardır, “Three Amigos” diye anılırlar: James Rumbaugh, Grady Booch, Ivar Jacobson. UML dili dışında 3 kitap, 1 el kitabı, 1 kullanıcı kılavuzu ve daha pek çok belge üretmişlerdir. OMG’deki çalışmanın çekirdeği bu üç kişidir.

Hoca, bir yöntem ürünleşmezse kimsenin onu kullanmayacağını vurgular. IBM’in desteğiyle Rational Rose, önce OMT’nin, sonra OMG’nin önerdiği diyagramları birebir ürünün içinde kullandırdı. Bugün bu diyagramları kullanan pek çok ürün vardır.

3.2 Analiz diyagramları ve tasarım diyagramları

UML diyagramları hem analizde hem tasarımda kullanılabilir. Ana ayrım şudur:

- **Analiz** çözülen problemle ilgilidir; problem uzayını modeller. Gerçek dünyaya daha yakındır.
- **Tasarım** problemin çözümüyle, nasıl çözüleceğinin teknik ayrıntılarıyla ilgilidir. Yazılım dünyasına daha yakındır.

Aynı diyagram türü kullanılsa bile analizde kavramsal sınıflar, tasarımda yazılım sınıfları yer alır.

UML’in sürümleri 1, 1.5, 2 diye ilerler. Derste UML 2 ele alınır. UML 2’de 14 çeşit diyagram vardır. Hepsi tanıtılacaktır, ama bir mimariyi ifade etmek için kullanılacak olan 3-4 tanesidir ve bunlara odaklanmak yeterlidir.

4. Yapısal ve davranışsal diyagramlar; diyagram kullanmanın faydası ve zararı

4.1 İki ana kategori

UML diyagramları iki ana kategoriye ayrılır.

Yapısal diyagramlar statiktir. Sistemin her zaman var olan parçalarını, bileşenlerini ve bunlar arasındaki ilişkileri tanımlar. Hoca bunu bir yapının planına (blueprint) benzetir.

Davranışsal diyagramlar dinamiktir. Sistemin nasıl çalıştığını, hangi işlevleri ve sorumlulukları olduğunu, bunların nasıl yerine getirileceğini gösterir. Hocanın benzetmesiyle, yapısal

diyagram bir makinenin tasarımıysa davranışsal diyagram o makineden paranın nasıl çıkacağını modeller. Dinamik davranışlar modellendiği için her davranış için ayrı bir diyagram çizilmesi gerekir; her diyagram farklı bir çalışma şeklini gösterir.

4.2 Diyagram kullanmanın faydaları

Hoca "diyagram kullanırsak ne olur, kullanmazsak ne olur?" sorusunu bir fayda-zarar analiziyle cevaplar. Faydalar üçtür:

- **İletişim:** Bilgi ekibin diğer üyelerine aktarılır. Bunlar ekip arkadaşları olabileceği gibi proje boyunca hiç konuşulmayan kişiler de olabilir; diyagramlar üzerinden daha kolay anlaşılır. Beş yıl sonra, belki siz orada yokken uygulamanın bakımını yapacak kişilere de bilgi aktarılmış olur.
- **Mevcut yöntemlere destek:** Kullanılan bir yazılım geliştirme metodolojisi varsa onu destekler. Örneğin bir tasarım gözden geçirmesi (design review) yapılacaksa diyagram yol gösterir, çünkü bütün sorumluluklar diyagramda gösterilmiştir ve kontrol edilebilir.
- **Araç desteği:** Diyagramlar elle de çizilebilir, ama genellikle CASE araçlarıyla çizilir. Bu araçlar eksik ve hataları yakalamaya yardım eder: yanlış bir sınıf bağlantısında uyarır ya da bir mesajın sorumluluğunun başka bir sınıfta olduğunu gösterir.

4.3 Diyagram kullanmanın zararları

Diyagramlar vakit alır; ürünü geliştirirken bir yandan da ürünün tanımları geliştirilir. Ayrıca sürüm ya da değişiklik yapıldıkça diyagramların da güncellenmesi gerekir. Sistemde bir şey değiştiğinde bütün diyagram ailesi gözden geçirilmeli, etkilenenler güncellenmelidir. Bu da ek bir iş gücüdür.

5. Yapısal diyagramlar

Hoca bu diyagramların bir kısmının bilindiğini, ama bilinmeyen ayrıntılar da olabileceğini söyleyerek hepsinin üzerinden somut örneklerle geçer.

5.1 Sınıf diyagramı

Sınıf diyagramı UML'in en popüler diyagramıdır.

> **Hocanın vurgusu:** Pek çok insan için "UML = sınıf diyagramı"dır, "UML çizdim" denince sınıf diyagramı kastedilir. Bu oldukça yanlış bir kullanımdır.

Sınıf diyagramına statik model de denir, çünkü sistemin statik yapısını gösterir. Eski "yapı diyagramı" adı da sınıf eklenerek "sınıf yapı diyagramı" biçiminde yaşatılır. Diyagram sistemin bileşenlerini, temel parçacıklarını gösterir. Temelde sınıflar ve ilişkiler ifade edilir. Bir sınıfta mutlaka olması gereken addır. Bunun dışında özellikler (attribute, oluşum değişkenleri) ve metotlar (davranışlar, işlemler) yer alır.

Sınıf diyagramında üç temel ilişki vardır: bağımlılık, ilişki (association) ve genelleştirme.

- **Bağımlılık** kesikli okla gösterilir ve "X, Y'yi kullanır" demektir. Bağımlılık her zaman görünür olmayabilir.
- **İlişki** (association), bir sınıfın başka bir sınıfla ilişki içinde olmasıdır: Y'yi etkilemek ya da Y'den bir örnek içermek gibi. İçinde barındırma durumu varsa ucuna eşkenar dörtgen konarak daha fazla ayrıntı verilir; buna birleşme (aggregation) denir ve "-den oluşur" anlamını verir.
- **Genelleştirme** "X bir Y tipindedir" demektir.

Hoca gizli bağımlılıklara dikkat çeker. X, Y ile; Y de Z ile ilişkili olsun. X, Y üzerinden Z'ye erişebiliyor ve Z'den bir şey kullanıyor ya da bir çağrı yapıyorsa, X aslında Z'ye bağımlı hâle gelmiştir. Bu bağımlılık diyagramda ayrı bir ilişki olarak mutlaka gösterilmeli, somutlaştırılmalıdır.

> **Hocanın vurgusu:** Genelleştirme tam olarak kalıtım değildir. UML'de modellemeden bahsedilir, gerçeklemeden değil. Programlama dilindeki `extends` ile birebir aynı sanıp geçmeyin; burada daha fazlası yapılabilir.

Slayttaki örnek sınıf diyagramında bir film, filmin fiyatı ve fiyatın alt sınıfları vardır. Çocuklar için fiyat (Children's Price), yeni çıkanlar için fiyat ve normal fiyat, Price'ın genelleştirmeyle bağlı alt sınıflarıdır: "çocuk fiyatı bir fiyat tipindedir". Ayrıca bir kiralama kavramı vardır; kiralama hem filmle hem müşteriyle ilişkilidir. Müşteriyle bir sözleşme yapılır ve sözleşmeler farklı türlerde olabilir. Yazılımı oluşturacak atomik bileşenler ve aralarındaki ilişkiler böyle gösterilir.

5.2 Nesne diyagramı

İkinci yapısal diyagram nesne diyagramıdır. Sınıflardan türetilmiş nesne örneklerini içerir; sistemin belirli bir T anında çalışırken olması gereken hâlini modeller. Slayttaki örnekte `c` nesnesi Company sınıfından, `d1` Department'tan türemiştir; `d1` ile `d2` ve `d3` ilişkilidir, `d3` bir Person nesnesi `p` ile ilişkilidir, Person'ın bir Contact Information'ı vardır. Nesnelerin adları, sınıfları ve o andaki özellik değerleri gösterilir. Hocanın ifadesiyle canlı sistemden belli bir fotoğraf çekip göstermek istiyorsanız bu diyagram kullanılır: sınıflar değil, o anda çalışan nesneler.

5.3 Kompozit yapı diyagramı

Slayttaki tanım şöyledir (ekrandan, 00:20:00):

- Daha az bilinir ve daha az kullanılır.
- Sınıfın iç yapısını göstermek için çizilir.
- Bizim için ilgili yanı arayüzlerdir. **Sağladığı arayüz** (provides interface): bu sınıf dünyaya bazı yetenekler sunuyor. **Gerektirdiği arayüz** (requires interface): bu sınıf dünyadan ne bekliyor?
- Sağladığı ve gerektirdiği arayüzler üzerinden sınıflar birbirine bağlanır.
- Bir yazılım mimarisinin parçalarını birleştirmenin yollarından biridir.

Slayttaki şekil metne şöyle dökülebilir:

```
cdClass
+-------------------------------+
|            Class              |
|   o-- Provides Interface      |
|   )-- Requires Interface      |
|        +--------+             |
|        |  Part  |          [] Port
|        +--------+             |
+-------------------------------+
```

Sağlanan arayüz yuvarlak uçlu (lolipop), gerekli arayüz yarım daire uçlu çizilir; hocanın benzetmesiyle prizler gibi birbirine takılırlar. Gerektirdiği arayüzler sağlanmadan sınıf çalışamaz. Mimaride arayüz kavramlarıyla modelleme yapılıyorsa kompozit yapı diyagramı kullanılabilir.

5.4 Bileşen (component) diyagramı

Bileşen diyagramı, mimariyi ifade eden diyagramların temelidir. Bileşenlerin nasıl bir araya geleceğini ve yapılarının statik olarak birbirine nasıl bağlanacağını gösterir.

Hoca UML referans el kitabındaki bileşen tanımını Türkçeye çevirerek verir: Bileşen, sistemin fiziksel ve değiştirilebilir bir parçasıdır. Gerçekleştirmeyi (implementasyonu) kapsar. Bir arayüz kümesine hem uyar hem de onun gerçekleştirmesini sağlar; yani hem bir işlevsellik sağlar hem de belirli bir arayüz şablonuna uymayı garanti eder.

Bileşen diyagramı özellikle kütüphanelerin ve kod varlıklarının modellenmesinde sık kullanılır: "Bu kütüphanede şu bileşenler var, şu hizmetleri hazır sunuyor, çalışmak için şunlara ihtiyaç duyuyor; bunları sağlarsanız bu hizmetleri alırsınız." Bileşenler arası ilişki, bir bileşenin diğerinin servislerini kullanmasıdır.

Slayttaki örnekte bileşen simgesi kenarında fiş gibi çıkıntılar olan bir kutudur. Ticket Database hem bir status hem bir purchase arayüzü sunar. Ticket Seller'ın üç arayüzü vardır: Subscription Sales, Individual Sales ve Group Sales. Aktörler de diyagramda yer alır (aktör, sistemde rol alan kullanıcıdır; adı rol almaktan gelir). Kesikli çizgiler bağlantı noktalarını gösterir. Group

Sales arayüzü yalnız Clerk'e, yani gişe görevlisine bağlıdır. Böylece kiosktan grup satışı yapılamayacağı modellenmiş olur.

5.5 Barındırma (deployment) diyagramı

Karmaşık sistemlerde parçalar ve bileşenler farklı işlem birimlerinde çalışabilir. Çalışma zamanındaki işlem birimlerinin nasıl yapılandırılacağı ve birbirleriyle nasıl etkileşeceği barındırma diyagramıyla gösterilir. Her düğüm (node) bir işlem cihazıdır; düğümler arasındaki kenarlar haberleşmeyi gösterir.

Slayttaki örnekte iki makine vardır. Sunucuda Scheduler bileşeni ve Meeting DB durur; istemcide Planner bileşeni vardır. İki makine doğrudan bir bağlantıyla (Direct Channel) bağlıdır. İstemcideki Planner, sunucudaki Scheduler'ın Reservation arayüzüyle haberleşerek işlemlerini yapar. Hoca, bu diyagrama bakınca bir sunucu katmanı ve bir istemci katmanı olduğunu, aralarındaki bağlantıları, her katmanda yazılımın hangi mimari parçalarının yer aldığını ve bunların birlikte nasıl hizmet verdiğini basitçe görebildiğini söyler.

5.6 Paket diyagramı

UML, Java'dan ya da .NET'ten bilinen paket yapısını destekler. Paketler genel amaçlı düzenleme mekanizmalarıdır; işlevselliği düzenlemeye yarayan bir organizasyondan başka bir şey değildir. Ad uzayı mantığıyla her paketin kendi ad uzayı olur: aynı adlı bileşenler farklı paketlerde çakışmadan kullanılabilir. Bir paket içindeki parçalar arasında başka paketlere bağımlılık varsa bu paketler arasında da gösterilmelidir.

UML 1 ve 1.5'teki gösterimde paket, kulakçıklı bir klasör ayracı gibidir; ad içine ya da kulakçığa yazılır. Paketler iç içe olabilir. Slayttaki örnekte Ordering bir alt sistem paketidir; içinde User Interface, Order Processing ve External Storage paketleri vardır. External Storage da Random Storage ve Stream Storage paketlerinden oluşur. Bunlar dışarıdaki başka paketlere bağımlıdır.

UML 2.0'da içerme gösterimi gelir. Örnekte Connection Sequence Controller açılmış durumdadır ve içinde iki bileşen daha vardır. Normal bir kullanım varsa (ör. kontrolör başka bir paketten bir şey kullanıyorsa) import gösterilir. İçerme, kontrolörü kapsayan paketin içine çizerek ya da "bunlar kontrolörün içinde yer alır" diye gösterilebilir.

5.7 Profil diyagramı

Yapısal diyagramların sonuncusu profil diyagramıdır; hoca bunu çoğunun duymadığını tahmin eder. UML, sınıflar ve ilişkiler gibi tanımlı parçalardan oluşan bir dildir. Bu parçaların tanımları değiştirilebiliyorsa UML dili de değiştirilebilir: UML'i UML kullanarak tanımlamak mümkündür. Buna UML'in meta modeli denir.

Tasarımcı kendi UML modelini genişletebilir; modeldeki belli öğelere yeni özellikler ya da yeni etiketler ekleyebilir. Bütün değişiklikler belirli bir UML profili içinde yapılır ve profil diyagramıyla ifade edilir. Bu genellikle meta sınıflar ve stereotip şablonlarıyla yapılır. Hocaya göre bu ileri seviye bir diyagramdır ve çoğu kişinin ihtiyacı olmayacaktır, ama standart UML yetmediğinde kendi profilinizi oluşturup profil diyagramıyla ifade etmeniz gerekir.

5.8 Yapısal diyagramların özeti

Slayttaki özet şöyledir (ekrandan, 00:30:00):

Diyagram	Gösterdiği
Sınıf	Bileşenler ve yapısal özellikleri
Kompozit yapı	İç yapı ve olası etkileşimler
Bileşen	Fiziki yazılım bileşenlerinin organizasyonu
Barındırma	Fiziki sistem kaynakları ve donanıma olan dağıtımları
Nesne	Belirli bir zamandaki statik yapı
Paket	Mantıksal gruplama ve bağımlılıklar
Profil	UML meta modeline yapılan eklentiler

Diyagram	Gösterdiği

Diyagramlarda fazla sayıda örtüşme bulunur: bir bileşen ya da sınıf pek çok diyagramda yer alabilir. Bu gayet normaldir.

Hoca son olarak hangi yapısal diyagramların mimariyi ifade ettiğini sorar. Sınıf diyagramı temel atomik parçaları, bileşen diyagramı bunları bir araya getiren hizmet yapısını, barındırma diyagramı fiziksel dağılımı gösterir; paket diyagramı da eklenince yazılımın mimarisinin statik yapısı ifade edilmiş olur.

6. Davranışsal diyagramlar

Yapısal diyagramlar sistemi bir bütün olarak, parçalarıyla tanımlar. Davranışsal diyagramlar ise bu parçaların nasıl davranacağını, her örneğin nasıl çalışması gerektiğini gösterir. Bir sınıf ya da bileşen diyagramıyla model ifade edilebilir, ama davranışlar için birden çok diyagram gerekir: her senaryo ve her sorumluluk için ayrı ardışıl ve işbirliği diyagramları çizilir. Yine bütün diyagram tiplerini kullanmak gerekmez; probleme en iyi katkıyı sağlayanlar seçilmelidir.

6.1 Kullanım senaryosu (use case) diyagramı

Use case diyagramı tek başına use case modellemesinin en önemli parçası değildir; sadece sonucun basit bir resmidir. Kullanım senaryosu modellemesinde en önemli iş senaryoları bulmak ve yazmaktır; işin metin tarafı çok daha fazladır.

OMT, analiz modelinde veri akış diyagramlarının kullanılmasını önerir. OMG ise bunun yerine Jacobson'ın kullanım senaryolarını, yani use case modellemesini ve görselleştirmesini önerir.

Kullanım senaryosu, kullanıcı tarafından gözlenebilen sistem eylemleri ya da etkileşim olaylarının toplamıdır. Sistemin bir kullanıcı etkileşimini baştan sona nasıl karşılayacağının ve nasıl bir hizmet sağlayacağının adım adım yazılmış hikâyesidir. Gereksinimleri ortaya çıkarmak için çok faydalıdır. Senaryolarla sistemin nasıl kullanılacağına dair hikâyeler çıkarılır, sonra dallanmalar somutlaştırılır: bir şey yanlış giderse ne olacak, açıkça söylenmeyen ara adımlarda ne olacak? Senaryoda tek cümle olarak geçen karmaşık bir adım için daha ayrıntılı bir "senaryocuk", yani bir sözleşme (kontrat) yazmak gerekebilir.

Diyagramın iki ana parçası vardır:

- **Çöp adamlar** (stick man): dış aktörler. Kullanıcı, başka bir sistem, başka bir cihaz olabilir; aktörün illa kanlı canlı bir insan olması gerekmez. Sistemle etkileşen bütün dış aktörler çöp adamla gösterilir.
- **Elipsler:** kullanım senaryoları; senaryonun adı elipsin içine yazılır.

  Hocanın vurgusu: Sık düşülen bir hata, use case diyagramını aktörlerden ve baloncuklardan oluşan bir hikâye gibi, her senaryo için ayrı ayrı çizmeye çalışmaktır. Diyagram, sistemdeki bütün senaryoların kümesinin bir arada gösterilmesidir. Diyagramdaki her elips için ayrı bir senaryo yazılmalıdır; sekiz senaryo varsa diyagramda sekiz elips beklenir. Her senaryonun içinde de pek çok adım ve cümle yer alır.

Aktörler ve senaryolar çizgilerle bağlanır. Açıklamasız bir çizgi ortaklık ya da katılım demektir: aktör o senaryoyla ilgilidir, onu kullanır, esas faydayı o sağlar. Bunun dışında iki özel bağlantı vardır:

- **`<<extends>>`:** Bir senaryo, zorunlu olmadan, duruma göre değişen biçimde genişletilir. Hocanın benzetmesi: "bir alana iki bedava" kampanyası. Normal akış da çalışabilir, ek parça da duruma göre devreye girer.
- **`<<include>>` ya da `<<uses>>`:** Bir alt program çağrısı gibidir. Pek çok senaryoda ortak kullanılan davranışlar ayrı bir senaryoya alınır. Örneğin kimlik doğrulama ayrı bir senaryoda çözülür ve yetkilendirme gereken her yerde o senaryo kullanılır.

Slayttaki diyagram metne şöyle dökülebilir (ekrandan, 00:37:20):

```
Aktörler ve bağlı senaryolar:
  Trading Manager   -- SetLimits
  Accounting System -- UpdateAccounts
  Trader            -- AnalyzeRisk, PriceDeal, CaptureDeal
  Salesperson       -- PriceDeal, CaptureDeal

Senaryolar arası ilişkiler:
  AnalyzeRisk   --<<uses>>-->    Valuation
  PriceDeal     --<<uses>>-->    Valuation
  LimitsExceeded --<<extends>>--> CaptureDeal
```

Hoca diyagramı şöyle okur: Trading Manager limitleri ayarlar ve bunu başka kimse yapmaz; limit ayarlamak sistemde bir işlevselliğin çalıştırılmasını gerektirdiği için bir senaryodur. Muhasebe sistemi hesapları günceller. Trader ve Salesperson (simsar ve satıcı) risk analizi ve fiyat teklifi gibi senaryolara erişir. Risk analizi ve fiyat teklifi bir değerleme (Valuation) içerir. CaptureDeal, Limits Exceeded ile birlikte de, tek başına da çalışabilir. Diyagramda 7 elips vardır, yani 7 senaryo yazılmalıdır. Hoca diyagramı "işlevselliğin tepeden çekilmiş fotoğrafı" olarak tanımlar: basittir ve taraflar arasında anlaşmayı kolaylaştırır.

6.2 Kullanım senaryosunun yazılması

Kullanım senaryosu bir hikâyedir; yapısız metin ya da tablo biçiminde yazılabilir. Genellikle cümle cümle maddelenerek yazılır. Giriş, aktörler, ön koşullar ve son koşullar gibi bölümleri de vardır; hoca bunların ayrıntısına girmez. Senaryolarda hep etken cümleler kullanılır: işi kim yapar, nerede yapar, neyle yapar açıkça yazılır.

Slayttaki örnek (ekrandan, 00:40:00):

- Ayşe Amazon'dan ürün almak ister.
- Ayşe Amazon web sitesine girer.
- Ayşe aradığı ürünü bulana kadar sitede gezer.
- Ayşe ürünü sepete ekler.
- Ayşe satın alma sayfasına geçer.
- Ayşe fatura bilgilerini verir ve ödemeyi tamamlar.

Hoca her cümlenin sistem için bir sorumluluk doğurduğunu gösterir. Ayşe'nin bir şey istemesi, modelde bir yere tıklaması gibidir. Ayşe'nin sitede gezmesi için sistemin ona ürün göstermesi gerekir. Bu sorumlulukların nasıl yerine getirileceği sözleşmelerle ifade edilebilir. Son cümle "ve" içerdiği için ikiye ayrılabilir: "Ayşe fatura bilgilerini verir" ve "Ayşe ödemeyi tamamlar". Sistemin bir onay ya da sipariş numarası üretmesi gibi ayrıntılarla senaryo zenginleştirilebilir; slaytta yer kaplamasın diye kısa tutulmuştur.

Tablo biçimi kullanılacaksa üç kolon gerekir: aktör kim, eylem nedir, nesne ya da nesneler nelerdir. Örneğin "Ayşe | ekler | Kitap, Sepet": kitap ve sepet nesneleri Ayşe'nin ekleme eyleminde etkileşime girmesi gereken nesnelerdir. Hocaya göre cümle cümle yazım daha çok kullanılır ve daha rahat okunur, ama tabloyu isteyen kullanabilir.

6.3 Bağlam (context) diyagramı

Bağlam diyagramı çok kullanılmaz ve aslında UML'in parçası değildir. Seviye 0 veri akış diyagramına (Level 0 DFD) eşittir. Bu diyagramda tek bir işlem birimi vardır, o da sistemin kendisidir. Sistemin dış paydaşlardan temelde ne aldığı ve onlara ne verdiği gösterilir: sistem kimlere hangi temel faydayı sağlıyor ve çalışması için mutlaka alması gereken temel girdiler neler?

6.4 Ardışıl (sequence) diyagram

En çok görülen ve kullanılacak davranış diyagramıdır. Hoca, birbirini mantık sırasıyla takip eden nesne ve olay zinciri olduğu için "ardışıl diyagram" demeyi tercih eder.

Slayttaki tanım (ekrandan, 00:42:30):

- Bir senaryoyu açıklar.

- Her kolonda tekil bir katılımcı (genellikle nesneler) yer alır.
- Zaman, aşağı doğru çizgisel olarak devam eder.
- Yatay çizgiler, nesneler arası mesajları gösterir.
- Telefon sanayisindeki mesaj sırası kartlarından evrimleşmiştir.
- İletişim diyagramlarıyla birebir aynı içeriğe sahiptir.

Ardışıl diyagramlar senaryolar için çizilir; her senaryodaki her sorumluluk ayrı bir ardışıl diyagram demektir. Kolonlardaki katılımcılar aktör de olabilir, ama genellikle nesneler görülür. Zaman T_0'dan T_N'e doğru aşağı akar. Kalın kutular metot gövdeleridir.

Slayttaki örneğin akışı:

```
c:Client        t:Transaction        p:ODBCProxy
   |--<<create>>-->|
   |--setActions(a,d,o)-->|
   |                      |--setValues(d,3,4)-->|
   |                      |--setValues(a,"CO")->|
   |<-- committed --------|
   |--<<destroy>>-->X
```

Hoca akışı şöyle anlatır: Client'ta bir metot çalışır (onu da birinin çağırması gerekir). Metot önce özel bir mesaj olan create mesajıyla Transaction sınıfından `t` nesnesini yaratır. Sonra `t`'nin `setActions` metodunu çağırır. Bu metot da kendi içinde ODBCProxy sınıfından türeyen `p` nesnesine değerler verir. İşlemin commit edildiği bildirilir ve Client `t` nesnesini yok eder (destroy, free). Client'ın bir senaryodaki bir eylemi yerine getirmek için yapması gereken bütün işlemler, zaman içinde nasıl akacağıyla birlikte tek bir diyagramda gösterilmiş olur.

6.5 İletişim (communication) diyagramı

İletişim diyagramı ardışıl diyagramla birebir aynı içeriğe sahiptir. Fark, birinin zaman içindeki akışı daha rahat görmeyi, diğerinin ise yer tasarrufu yapmayı sağlamasıdır. Haberleşmeler numaralarla gösterilir. Aynı örnekte Client Transaction'a, Transaction da Proxy'ye bağlıdır:

```
1    : create
2    : setActions
2.1  : setValues
2.2  : setValues
3    : destroy
```

2.1 ve 2.2, ikinci metot çağrısının (setActions) içinde yapılan çağrılardır. Numaralandırma, kitaplardaki bölüm ve alt bölüm (section, subsection) numaralandırması gibi 1.1, 1.1.1 biçiminde dallanan ondalık gösterimdir.

6.6 Aktivite diyagramı

Aktivite diyagramı Petri ağlarından türemiştir. Hoca burada bir durum geçiş makinesi tasarlandığını söyler.

> **Ek bilgi (derste anlatılmadı):** Petri ağı, yerler (place), geçişler (transition) ve aralarında hareket eden jetonlarla (token) eşzamanlı sistemleri modelleyen matematiksel bir gösterimdir. UML 2'de aktivite diyagramının anlamı bu jeton akışı fikrine dayandırılmıştır. Bu yüzden aktivite diyagramı paralel akışları ve senkronizasyonu doğal olarak gösterebilir.

Paralel çalışan işlemler varsa ve bunların belli noktalarda senkronize olması gerekiyorsa bu ardışıl ya da iletişim diyagramlarıyla gösterilemez. Aktivite diyagramı iş akışını, işlemlerin nasıl senkronize edileceğini ve eşzamanlılığın nasıl sağlanacağını gösterir.

Slayttaki örnekte akış içi dolu başlangıç noktasından başlar ve bir sipariş hazırlama (Setup Order) işlemine gelir. Sonra bir karar vardır: tekil sipariş (Single Order) mı, abonelik (Subscription) mi? Tekil siparişse bir yoldan devam eder ve birleşme noktasına gelir. Abonelikse siyah bir çubuğa (bar) gelinir. Bu çubuk bir senkronizasyon noktasıdır; burada paralelleşme başlar. Örneğin ödül ya da bonus kazandırma için ayrı işlemler çalışır; her biri belki bir ardışıl diyagramla modellenmiş bir senaryo gerçeklemesidir. Bir kol 10 saniyede, öbürü 3 saniyede

bitebilir. Erken biten, ikinci bara gelip diğerini bekler. İkinci bar ikinci senkronizasyon noktasıdır: paralel kollar burada yeniden birleşir ve akış sonuçlarla birlikte devam eder. Karar noktasından ayrılan yollar da bir eşkenar dörtgende birleşip aşağı akar.

6.7 Genel etkileşim (interaction overview) diyagramı

Genel etkileşim diyagramı bir çeşit aktivite diyagramıdır. Başlangıçtan sonra gelen düğümlerin her biri alt seviyede bir etkileşim diyagramıdır. Örnekte önce bir arama senaryosu çalışır. Bir şey bulunursa başka bir sorumluluğa geçilir, bulunmazsa arama tekrarlanır. Bulunduğunda Checkout senaryosu çalışır; sonra duruma göre iki senaryodan biri çalışır ve biter. Böylece aktivite diyagramı daha üst seviyeye çıkarılıp genelleştirilmiş olur. Düğümler ardışıl diyagram, iletişim diyagramı, başka genel etkileşim diyagramları, hatta zamanlama diyagramları olabilir; yani UML'deki diğer davranış diyagramlarıdır.

6.8 Zamanlama (timing) diyagramı

Zamanlama diyagramı da çoğu kişinin duymadığı diyagramlardandır ve dijital çip tasarımından mirastır. Sistemdeki durumların belirli zamanlamalara uyması gerekiyorsa kullanılır. Zaman soldan sağa ilerler; slayttaki örnekte milisaniye cinsinden 0'dan 110-120 ms'ye kadar olaylar vardır. Oklar senkronizasyon noktalarını gösterir: "şu, şuraya senkronize". Çok kullanılmaz, ama belirli zamanlamalar gerekiyorsa bu diyagramla modellenir. Hocanın tarifiyle bir çeşit döndürülmüş ardışıl diyagramdır: zaman aşağı doğru değil, yana doğru gider.

6.9 Durum (state) diyagramı

Slayttaki tanım (ekrandan, 00:50:00):

- Davranışsal diyagramların en güçlü ve karmaşık olanıdır.
- Sonlu durum makinelerinin yeteneklerini birleştirme (aggregation), eşzamanlılık, geçmiş ve olayları tüme yayım gibi maddelerle genişletir.
- Bunlarla ilgili ayrı bir ders yapılacaktır.

Basit örnekte iki dış durum vardır: sistem ya Idle ya da Maintenance durumundadır. Aralarındaki geçiş `maintain` koşuluyla olur: `maintain` doğru olduğunda sistem Maintenance durumuna geçer. Bu durumun içinde, kesikli çizgiyle ayrılmış, eşzamanlı çalışan iki alt durum vardır. Biri Testing'dir: başlar, sırayla işlerini yapar (bunlar da ardışıl diyagramlarla ifade edilebilir) ve biter. Diğeri Commanding'dir: tuşa basıldığı (keypress) sürece bekle, komut işle, bekle, komut işle döngüsüyle devam eder ve sonunda biter. Durum içinde durum, onun içinde paralel durumlar ve geçmiş modellemesi gibi ayrıntılar ilerideki derse bırakılır.

6.10 Davranışsal diyagramların özeti

Hoca davranışsal diyagramları da toplar; böylece 7 + 7 = 14 diyagram tanıtılmış olur:

- **Aktivite:** aktiviteler arasındaki akış kontrolü, senkronizasyonla birlikte.
- **Ardışıl ve iletişim:** birbirinin aynısı, farklı gösterimler. Nesnelerin nasıl etkileşeceğini ve sorumlulukların nasıl yerine getirileceğini, yani ayrıntılı tasarımı gösterir.
- **Genel etkileşim:** alt seviyedeki aktivite diyagramlarının birleştirilmesi.
- **Zamanlama:** zamanın yana doğru aktığı, döndürülmüş bir ardışıl diyagram; zaman senkronizasyonu olan işler için.
- **Use case:** analizin diyagramı; sistemin dış aktörlere sağladığı işlevsellik ve sorumluluklar.
- **Durum:** belirli uyaranlara tepki olarak hangi durumda olunacağı ve nasıl dinamik davranış sergileneceği.

Hoca bunların hepsine bir anda ihtiyaç duyulmayacağını, en çok ardışıl diyagramların kullanılacağını, zamanlama gibi özel ihtiyaçlar olduğunda ilgili diyagramın ayrıntısının öğrenilip kullanılabileceğini söyler.

7. Nesne Kısıt Dili (OCL) ve UML meta modeli

7.1 OCL

UML hep diyagramlarla ilerler, ama bazı şeyler diyagramla ifade edilemez. Belli kısıtlar başka bir dille gösterilmelidir. Bunlara UML'in diyagramsız kısmı ya da diyagramların metin uzantısı denebilir: OCL (Object Constraint Language, Nesne Kısıt Dili). Genellikle sınıflarda ve durum geçiş diyagramlarında kullanılır. Sistemde mutlaka kesin tanımlanması gereken şartlar varsa OCL ile yazılır.

Slayttaki örnek (ekrandan, 00:54:30): Sınıf diyagramında bir hesap sınıfı var. Hesap sınıfının para yatırma işlemi olsun. Bu işlem "miktar" adında bir gerçek sayı alsın. `pre` anahtar kelimesi miktarın sıfırdan büyük olmasını, `post` anahtar kelimesi son miktarın mevcut + ilk miktar olmasını gösterir.

```
context Account::deposit(Real : amount)
pre:  amount > 0
post: balance = balance@pre + amount
```

Hoca satırları şöyle açıklar. `pre`, metodun çalışması için mutlaka sağlanması gereken koşuldur: sıfırdan küçük değerler için metot çalışmaz ve bu garanti edilmelidir. `post`, işlem bittiğinde sistemin garanti ettiği durumdur: yeni bakiye, bakiyenin başlangıçtaki hâline (`balance@pre`) verilen pozitif miktarın eklenmesidir. Başka herhangi bir davranış olmayacaktır.

> **Ders sonrası not:** Slayttaki parametre `Real : amount` biçiminde yazılmıştır. OCL ve UML'de parametre önce ad, sonra tip olarak yazılır: `context Account::deposit(amount : Real)`.

Hocanın bir başka örneği: 18 yaşından küçükler için bir davranışın çalışmaması gerekiyorsa bu da OCL ile tanımlanabilir. Sınıf diyagramında metodun yanına "18'den küçükse lütfen çalıştırmayın" diye not düşmek yerine kısıtın formal tanımı yapılır.

7.2 UML meta modeli

Hoca meta modeli hatırlatır: meta model UML'i UML ile tanımlamaktır ve profil diyagramlarıyla yapılır. OCL'de kısıtlar modele değil, diyagramlara konur; meta modelde ise UML'in kendisi özelleştirilir. Slayttaki meta model parçasında Class'ların association'ları, attribute'ları ve operation'ları vardır; Class bir Classifier'dır; attribute'lar primitive data type'lara sahiptir; operation'ların parametreleri vardır. Bu parçalardan bir şey değiştirilmesi gerekiyorsa profil diyagramıyla meta model değiştirilir. Hoca bunu "ileri mimarinin ilerisinin de ilerisi" olarak niteler ve ilgilenenleri UML profilleme konusunu araştırmaya davet eder.

7.3 İlk saatin sonucu

Karmaşık bir sistemde parçaların ne yapacağı varsayılmadan tasarım gerçekleştirilemez. Diyagramlar ifadenin temsil biçimidir: problemi nasıl anladığınızı ve nasıl çözdüğünüzü gösterir. Hocaya göre bu sayede daha az problemle boğuşarak yola devam edilir; modellemenin ve mimari tasarımın önemi buradadır.

8. Nesneye yönelik analiz: neden nesneler?

8.1 Tasarıma nereden başlanır?

Aradan sonra hoca temel soruyu sorar: tasarıma nereden başlanır? Problemi çözmeden, yani tasarlamadan önce onu anlamak gerekir. Problemin anlaşıldığı ve "bunu anladım" diye gösterildiği aşamaya analiz aşaması denir.

Nesne analiz modeli yaratırken doğal dille tanımlanan gerçek dünya nesneleri yazılım uzayına taşınmaya çalışılır. Böylece kavramlar daha iyi ifade edilir ve yazılım gerçek dünyaya yakınlaştırılır. Bunlar sınıf diyagramıyla ifade edilecektir. Hoca, yöntemin 1980'lerde Abbott ve bir çalışma arkadaşı tarafından geliştirildiğini söyler.

> **Ek bilgi (derste anlatılmadı):** İsimlerden sınıf, fiillerden işlem çıkarma tekniği büyük olasılıkla Russell J. Abbott'un 1983 tarihli "Program Design by Informal English Descriptions" makalesine dayanır. Teknik daha sonra Grady Booch'un nesneye yönelik tasarım çalışmalarıyla yaygınlaşmıştır.

Nesneye yönelik analiz yapılmayacaksa alternatif fonksiyonel çözümlemedir; hoca buna "eski usul" der. Bu yaklaşımda "sistem hangi uyarana hangi tepkiyi vermeli, kullanıcıya nasıl hizmet sağlamalı, hangi sorumlulukları olmalı?" soruları yerine "sistemin hangi işlevleri olmalı, dışarıya hangi fonksiyonları hizmet olarak sunmalıyım?" sorularına odaklanılır. Bu da olabilir, ama bu derste nesneye yönelik yoldan gidilecektir.

8.2 Neden fonksiyon yerine nesneler?

Slayttaki gerekçe şöyledir (ekrandan, 01:20:00):

- Yazılım sistemlerinin uzun yaşam ömürleri vardır ve bu süre boyunca evrimsel değişikliklere uğrarlar.
- Özellikler eklendikçe, hatalar düzeltildikçe, kütüphaneler değiştikçe programın orijinal yapısı da kaybolur.
- Bakım esnasında fonksiyonlar nesnelerden çok daha sık değişir.
- Örnek, bankacılık uygulaması: faiz hesabı, işlem ücreti hesaplama, vergi alma gibi fonksiyonlar zaman içinde değişir. Hesaplar, müşteriler, işlemler gibi nesneler ise değişmez.
- Nesneler fonksiyonlardan daha kararlı yapılardır. Nesneler daha sürdürülebilir tasarımlar sağlar.

Hoca değişimin kaynaklarını sayar: istekler, şartlar, altyapı ve donanım değişebilir; hizmet verilen kitle büyüyebilir. Bankacılık örneğinde faiz ve vergi düzenlemeleri kısa sürede birkaç kez değişebilir. Bunlar fonksiyon olarak modellenseydi fonksiyonlar sürekli değişmek zorunda kalırdı. Oysa hesap ve müşteri kavramları değişmez; hizmet aynıdır, yalnız hizmetin içeriği, yani işlevselliği değişir. Nesneler kavramsal oldukları ve gerçek dünyada karşılıkları bulunduğu için daha az değişir; bu yüzden nesneyle yapılan tasarım daha sürdürülebilirdir ve çözümün uzun süre yaşamasını sağlar.

8.3 Yapısal analiz ile nesneye yönelik analizin farkı

Yapısal analiz ve tasarımda işlevselliğe, fonksiyonları bulmaya, neyin hesaplanacağına odaklanılır; veriler ikinci plandadır. Nesneye yönelik analizde ise veri nesneleri ön plana alınır. Önce kavramlar, özellikleri ve özelliklerin veri tipleri modellenir, davranışlar sonradan eklenir. Hocaya göre ikisi aynı noktadan başlıyor gibi görünse de tamamen farklı, hatta zıt yönlerde ilerleyen iki analiz modelidir.

9. Dilbilgisinden modele: ağaç yapraklarını sayma örneği

9.1 Yöntem

İlk iş sistemin gereksinim belgelerini, yani sistemin metin ifadesini almaktır (ör. bir SRS belgesi). Yoksa analist bunu oluşturur. Sonra senaryolar (use case'ler) ele alınır ve içlerindeki isim, fiil, sıfat gibi kelime türleri bulunup altı çizilir; hocanın deyişiyle ortaokuldaki dil bilgisi dersine geri dönülür.

- **İsimler** sınıfları verir.
- **Sıfatlar** özellikleri verir.
- **Eylem bildiren fiiller** işlemleri verir.
- **Durum bildiren fiiller** ilişkileri verir.

Ortaya çıkan sınıf modeli müşteriyle birlikte gözden geçirilir ve adım adım, yinelemeli olarak ilerlenir. Hoca "Kolaymış hocam, isim, sıfat, fiil…" itirazını kendisi dile getirir ve cevaplar: düşünerek çözülmesi gereken pek çok nokta olacaktır.

9.2 Problem ve senaryo

Örnek, bir ağaçtaki yaprak sayısını saymaktır. Slayttaki metin şöyledir (ekrandan, 01:24:45):

Ağaç: Döngü içermeyen çizge. Her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.

Yöntem:

1. Sayılmamış ağaç parçalarını alıp boş yığına koyun.
2. Yaprakları sayacını sıfır yapın.
3. Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.
4. Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.
5. İki tane alt ağaç varsa sol ve sağı ayırıp ikisini de yığına ekleyin.
6. Yığın boşaldığında, yaprak sayısını gösterin.

Hoca tanımı açıklar: ağaç özel bir çizgedir, farkı kapalı döngü (çevrim) içermemesidir. Bir düğüm en fazla bir yerden yukarıya bağlıdır, ama altında birden fazla düğüm olabilir; ikili ağaç gibi değildir. Altında hiç düğüm olmayan düğüm yapraktır, yani son düğümdür.

Algoritmayı da anlatır: sayılmamış ağaç parçaları boş bir yığına konur, yaprak sayacı sıfırlanır. Yığın boş olmadığı sürece yığından bir ağaç alınıp incelenir. Ağacın çocuğu, yani alt ağacı yoksa o bir yapraktır: sayaç artırılır ve parça atılır, yığına bir daha konmaz. İki alt ağacı varsa sol ve sağ ayrılıp ikisi de yığına eklenir ki onlar da incelenebilsin. Bütün parçalar incelenip atıldığında sayılacak parça kalmamıştır ve sayaç yaprak sayısını gösterir. Hoca bunun senaryonun ana akışı gibi düşünülebileceğini söyler.

> **Ders sonrası not:** Slayttaki tanım bir düğümün "sıfır ya da daha fazla" alt düğümü olabileceğini söylüyor ve hoca ağacın ikili ağaç olmadığını belirtiyor. Ancak yöntem yalnız iki durumu ele alıyor: tek yaprak ya da iki alt ağaç. Genel bir ağaçta 5. adımın büyük olasılıkla "bütün alt ağaçları yığına ekleyin" biçiminde olması gerekir. Bu fark analiz modelini etkilemez; örnek, senaryonun ana akışını göstermek içindir.

9.3 Adım 1: İsimlerden aday sınıflara

Hoca senaryodaki isimleri tek tek işaretler: ağaç parçaları (isim tamlaması), yığın, boş yığın ("boş" sıfattır), yapraklar, sayaç, sıfır (o da bir isimdir), ağaç, tek yaprak, sol ağaç, sağ ağaç, alt ağaç, yaprak sayısı. "Yığın boşaldığında" ifadesi yığının bir boşalma durumu ya da özelliği olduğunu gösterir; "yaprak sayısını gösterin" de sayaçtaki değere işaret eder.

İşaretlenen kelimeler: yığın, parçalar, ağaç, alt ağaç, sayı, sayaç, yaprak, yapraklar, sıfır. Sonra temizlik yapılır:

- Bazı kelimeler tekrar eder ya da kökleri ortaktır: ağaç, alt ağaç, sol ağaç, sağ ağaç. Yaprak ve yapraklar aynı şeydir.
- Sayaç ile sayı tek bir şey mi, ayrı şeyler mi? Bunun düşünülerek karar verilmesi gerekir.
- **Yığın** önemli görünür: ona bir şey konur, ondan bir şey çekilir. Aslında bir yığıt (stack) yapısıdır, bu yüzden bir kavram olarak tutulmalıdır.
- **Parça** aslında bir ağaç ya da alt ağaçtır; "parça" yerine "alt ağaç" demek kavramı daha iyi açıklar.
- **Ağaç** önemlidir, kalır. **Sayaç** da kalır, çünkü bir şeyler sayılıp artırılacaktır.
- **Yaprak**, altında alt ağaç olmayan ağaçtır; aslında özel bir ağaç adlandırmasıdır.
- Analist, senaryoda geçmeyen bir kavramı da ekleyebilir. Ağaçların düğümleri olduğunu bilen analist "düğüm" kavramını ekleyebilir; bu kavram alt ağaç ve yaprak gibi kavramların yerini alabilir ve eksikleri görmeyi kolaylaştırır.
- **Sıfır** bir sınıf değildir, ama modelde bir yeri vardır: bir özelliğin ilk değerinin sıfır olması gerektiğini ifade eder.

  > **Hocanın vurgusu:** Sıfırı silerken tamamen atmayın; ileride bir özelliğin ilk değeri olarak kullanacağınızı unutmayın.

Aday sınıflar, daha genel ya da daha iyi anlatan, adı daha düzgün kavramlar altında gruplanır. Sonuçta dört sınıf kalır (ekrandan, 01:30:00): **Counter** (sayaç), **Leaf** (yaprak), **Tree** (ağaç) ve **Pile** (yığın). Pile aslında Stack'tir; adı farklı olsun diye Pile seçilmiştir. Sınıf, birbiriyle ilgili nesne grubunun tanımıdır.

Hoca burada önemli bir uyarı yapar: hiçbir sonuç kesin değildir. "Benimki doğru, başka bir model çıkarırsanız yanlış" gibi bir durum yoktur. İlk derslerde söylendiği gibi tasarım öğretilemez, ancak öğrenilebilir; yaptıkça sonuçlar gerçek dünyayı daha iyi modeller. Problem hakkında

bilgi arttıkça analiz modeli bile sürekli revizyona uğrar. Analist gereksinimlerdeki eksikleri gidermek ve müşterinin gerçekten istediği modeli kurmak için sık sık müşteriye dönmelidir. Müşteriler çoğu zaman sorulan şeyleri düşünmemiştir bile; "Öyle mi, o da mı oluyordu?" diye cevap verebilirler ve çözüm beklediklerinden fazlası olduğunda teşekkür ederler. Bu artımlı bir süreçtir; tek seferde doğru analiz modeli çıkmaz. Ortak anlaşılan bir noktada problem uzayının yeterince modellendiğine karar verilip durulmalıdır.

9.4 Adım 2: Sıfatlardan özelliklere

Sıfatlar, etkiledikleri isimlerle birlikte işaretlenir: boş yığın, yaprakların sayacı, tek yaprak, sol ağaç, sağ ağaç, iki ağaç. Her sıfat özellik olmak zorunda değildir; elenebilir. İsim tamlamalarından da (ağaçların parçaları, yaprakların sayısı) özellik çıkarılabilir.

- "İki ağaç" zaten sol ve sağ ağaçla karşılanır; bu yüzden sol ağaç ve sağ ağaç özellikleri yeterlidir.
- "Tek yaprak" yaprakla ilgilidir.
- "Parça" ağacın bir parçasıdır, yani ağacın kendi içindedir; büyük olasılıkla ileride kaldırılacaktır.
- Sayacın içinde bir sayı vardır.
- Yığının boş olup olmadığı sorulabilmelidir; bu yüzden "boş olma" durumu bir özellik olarak eklenir (true/false tutan bir `is empty` gibi).

Güncellenmiş sınıf modeli (ekrandan, 01:33:00):

```
Counter: count
Leaf:    (özellik yok)
Tree:    left subtree, right subtree, leaf/leaves, part
Pile:    empty
```

9.5 Adım 3-4: Fiillerden işlemlere ve sorumlulukların atanması

İşlemler eylem fiillerinden bulunur: nesnelerin sağladığı, tam söylemek gerekirse bilişimsel servislerdir. Durum fiilleri ise nesneleri değil durumları bildirir ve sınıflar arasındaki çizgileri, yani ilişkileri oluşturur.

Senaryodaki fiiller: saymak, koymak, yapmak, almak, incelemek, arttırmak, atmak, ayırmak, eklemek, göstermek. "Sayılmamış" bir eylem değil, bir durumdur (sayılmış ağaç, sayılmamış ağaç), ama bir sayma işi vardır.

Asıl zor soru, hangi işi kimin yapacağıdır:

- **Yığını kim tutacak?** Her sistemde `main` metodunun bulunduğu bir ana sistem sınıfı vardır. Bu sınıf genellikle modelde gösterilmez, ama gerçekleştirmede başlangıç için mutlaka gerekir.
- **Saymayı kim yapacak?** Sayaç mı yaprakları saysın, ağaç mı kendini saysın? Hocaya göre bu tamamen tasarımcının elindedir. Sorumluluk sayaca verilebilir, çünkü adı gereği bir şeyleri sayması beklenir. Ağaca da verilebilir, ama o zaman ağaç kendi içinde gereksiz işler yapmaya başlar. Karar verirken ilk haftalarda anlatılan iyi uyum ve az bağımlılık ilkeleri gözden geçirilmelidir.
- **Yığına kim koyacak, kim çekecek?** Push ve pop doğal olarak yığının sorumluluğudur. Yığın argüman olarak aldığı ağaç nesnesini koyar. Adlar (push/pop, insert/extract, put/get) tasarımcıya kalmıştır.
- **Sayacı kim sıfırlayacak, kim artıracak?** Kendi işinin uzmanı olarak sayacın kendisi.
- **Ağacı kim inceleyecek?** Sistem mi, yığın mı? Bu da tasarımcıya kalmıştır. Sorumluluğu hemen atamak zorunlu değildir; davranışlar ileride modellenirken bu ayrıntı eklenebilir.
- **Ağacı kim yok edecek, kim sol ve sağ diye bölecek?** Ağacın kendini bölmesi ve kendini yok etmesi daha mantıklıdır.

Bilinebiliyorsa işlemlerin parametreleri, veri tipleri ve dönüş değerleri de belirtilebilir; ya da analiz için bu kadar ayrıntının yeterli olduğuna karar verilebilir. Hoca esas amacı tekrarlar: problemin parçaları nelerdir, hangi gerçek dünya kavramları yazılım olarak modellenmelidir, bunlar hangi özelliklerle gösterilir ve hangi servisleri sunar?

9.6 Adım 5: Durum fiillerinden ilişkilere

Üç çeşit bağlantı hatırlatılır: genelleştirme, birleştirme ve ilişki.

Genelleştirme (ya da tersten okununca özelleştirme), alt sınıfın nesnelerinin üst sınıfın türünde olmasıdır. Ağaç daha genel, kapsayıcı bir kavramdır, üst kümedir; yaprak (Leaf) onun özelleşmiş hâlidir. Yaprak normalde ağacın içinde de modellenebilir, ama özel bir alt küme olduğunu göstermek gerekiyorsa genelleştirme ilişkisi kullanılır. Hocanın diğer örneği: araç üst sınıftır; araba, otobüs, motor yat, uçak birer araçtır.

Birleşme (aggregation), parça-bütün ilişkisidir; bir çeşit "X koleksiyonu, X grubu". Pile yalnız ağaçları tutan özel bir yığıttır: "Pile consists of Tree". Ayrıca "Tree consists of Tree": ağacın alt ağaçları, parçaları vardır. Bu kavram da böylece modellenmiş olur.

İlişki (association) durum bildiren fiillerden çıkar: kim kimle konuşacak, kim kime hizmet verecek, kim kimin metodunu çağıracak? Bağlantılar adlandırılarak anlamlı hâle getirilir. Hocanın örneği: "Öğrenci üniversitede okur" cümlesinden Öğrenci ve Üniversite arasında bir "okur" ilişkisi çizilir. Ağaç örneğinde "sayaç yaprakları sayar", yani Counter ile Leaf arasında bir ilişki vardır.

Son sınıf modeli (ekrandan, 01:40:00):

```
Sınıflar
  Counter : özellik -count            işlemler -increment, -display
  Leaf    : (boş)
  Tree    : özellik -left subtree, -right subtree, -leaf/leaves, -part
            işlemler -split, -throw away
  Pile    : özellik -empty
            işlemler -put a Tree onto, -get a Tree from, -take a tree off

İlişkiler
  Counter ----counts---- Leaf          (ilişki, association)
  Leaf    ----|>         Tree          (genelleştirme: Leaf bir Tree'dir)
  Pile    <>----         Tree          (birleşme: Pile ağaçlardan oluşur)
  Tree    <>----         Tree          (kendine birleşme: ağaç ağaçlardan oluşur)
```

Özetle: Counter `counts` ilişkisiyle Leaf'e bağlıdır; Leaf, Tree'nin alt sınıfıdır; Pile, Tree'lerden oluşur (eşkenar dörtgen Pile tarafında); Tree de Tree'lerden oluşur (kendine birleşme).

Hoca bu modelin de tek doğru olmadığını vurgular: kimse tam doğru ya da yanlış diyemez. Bir başkası Leaf'i hiç koymayıp Tree'nin yeterli olduğunu söyleyebilir ve Counter'ı doğrudan Tree'ye bağlayabilir; bu da bir çözümdür. Hoca, yaprağın özel bir şeyleri olduğunu düşündüğü için özelleştirmeyi göstermeyi seçmiştir. Her şey bir sistem sınıfından ("ağaç yaprak sayma sistemi") başlar, ama bu sınıf genellikle diyagramlarda gösterilmez.

9.7 Gerçek gereksinim belgeleri

Hoca örneğin sınırlarını da belirtir. Tipik gereksinim belgeleri altı cümleden oluşmaz; burada tek bir davranış ve dört kavram vardı. Gerçek belgeler yüzlerce sayfadır ve pek çok özel terim içerir. Ayrıca örnekte algoritmik bir çözüm ayrıntısı paylaşıldı; normalde gereksinim belgelerinde böyle algoritmik çözüm yolları değil, yalnız problem tanımları bulunur. Senaryoların yazılması da çoğu zaman analize kalır ve mutlaka yapılması gerekir.

10. UML sınıf modelinin ayrıntıları

10.1 Sınıf diyagramında neler bulunur?

Sınıf modelleri UML'in en popüler tarafıdır. Analiz modeli kavramsal sınıf diyagramıyla (conceptual class diagram) rahatça ifade edilir, ama bu model aynı zamanda karmaşıktır: neyin ne kadar ayrılacağı, kimin nerede durup nasıl bağlanacağı çoğu zaman hissederek bulunur. Henüz var olmayan bir şeyin resmi çizilir.

Sınıf diyagramına statik yapı diyagramı da denir. Sınıfların dışında arayüzler, nesneler ve nesneler arası ilişkiler de yer alabilir. Ayrıntılar için temel kaynak UML referans el kitabıdır.

Sınıf, benzer örneklerin küme tanımı, yani şablon tanımıdır. Bir alandaki (domain) nesneler bir grup oluşturuyorsa ve uygulama için birden fazla anlamlı nesne türetmek mantıklıysa bunlar sınıf olabilir. Roller, olaylar, hatta etkileşimler de sınıf olabilir: kendi özellikleri ve davranışları olan etkileşimler ayrı bir sınıf olarak modellenir, yani sınıflar arasındaki çizgi de bir sınıfa dönüştürülebilir.

10.2 Sınıf kutusu ve soyut sınıf

Sınıf genellikle üç bölmeli bir dikdörtgenle gösterilir. Olmazsa olmaz bölme addır (ör. Window). Özellikler ikinci, davranışlar üçüncü bölmeye yazılır. Sorumluluklar, aykırı durumlar ve aktarılmak istenen başka bilgiler varsa bunlar da ayrı bölmelerde gösterilebilir.

Ad eğik yazılmışsa ya da `<<abstract>>` stereotipiyle verilmişse sınıf soyuttur. Soyut sınıf bir şablon kontratıdır: kendisinden asla örnek türemez, ama alt sınıfların özelliklerini tanımlar. Ortak özellikleri olan ilişkili sınıflar varsa soyut sınıf kullanmak mantıklıdır.

Stereotipler (hocanın tercih ettiği Türkçe karşılıkla “klişeler”) temel modelleme dilini genişletmeye yarar. Meta modele eklenerek yeni stereotipler yaratılabilir; bu ileri modelleme gerektiğinde kullanılacak bir araçtır.

10.3 Özellikler alanı

Sınıflar gerçek hayat nesnelerinden yola çıkarak mümkün olduğunca yazılıma taşınır. Nesnenin nitelikleri özellik (değişken), davranışları metot olarak gösterilir. Slayttaki kurallar (ekrandan, 01:47:00):

- Tüm özelliklerin adları, tipleri ve erişim türleri vardır.
- Erişim türleri görünürlükleri (diğer sınıflardan erişilebilirliklerini) tanımlar. Başta opsiyoneldir, sonra eklenebilir.
- Çoğullamalar da belirtilir.
- Özelliklerin tipleri tanımlanmalıdır.
- İlk değer atamaları istenirse yapılabilir. Türetildiği (hesaplanarak bulunduğu, atanmadığı) da belirtilebilir.
- Özel tanımlamalar `{..}` ile yapılabilir: `{frozen}` özelliğin değerinin değiştirilemeyeceğini gösterir.

İşaret	Görünürlük	Açıklama
+	Genel	Diğer sınıflara görünür
-	Özel	Sınıfın kendi içinde saklanır
#	Korumalı	Yalnız alt sınıflar (sınıfı genişletenler) görebilir
~	Paket	Aynı paketin içinden görünür, başka paketlerden görünmez

Hoca, erişim türleri başta opsiyonel olsa da görülebildiği kadarının eklenmesini önerir: model ne kadar ayrıntılıysa o kadar faydalıdır ve ileride değiştirilebilir. Çoğullama, bir özellikten kaç tane olacağını (sıfır, bir, en az bir, n tane) gösterir.

> **Hocanın vurgusu:** Özellik tipleri için mümkün olduğunca basit veri tipleri kullanın. Tip karmaşıksa orada aslında bir birleşme (aggregation) ya da bütünleşme (composition) yapısı vardır; o tipi ayrı bir sınıf olarak modellemeniz gerekir.

İlk değer örneği ağaç modelindeki sayacın sıfırlanmasıdır. Türetilmiş özellik örneği ise faizdir: doğrudan değer atanmaz, hesaplanarak bulunur. Önceki bir slaytta yazar ve test durumu gibi özel tanımlar da küme parantezleri içinde verilmişti.

10.4 İşlemler alanı

Slayttaki kurallar (ekrandan, 01:50:00):

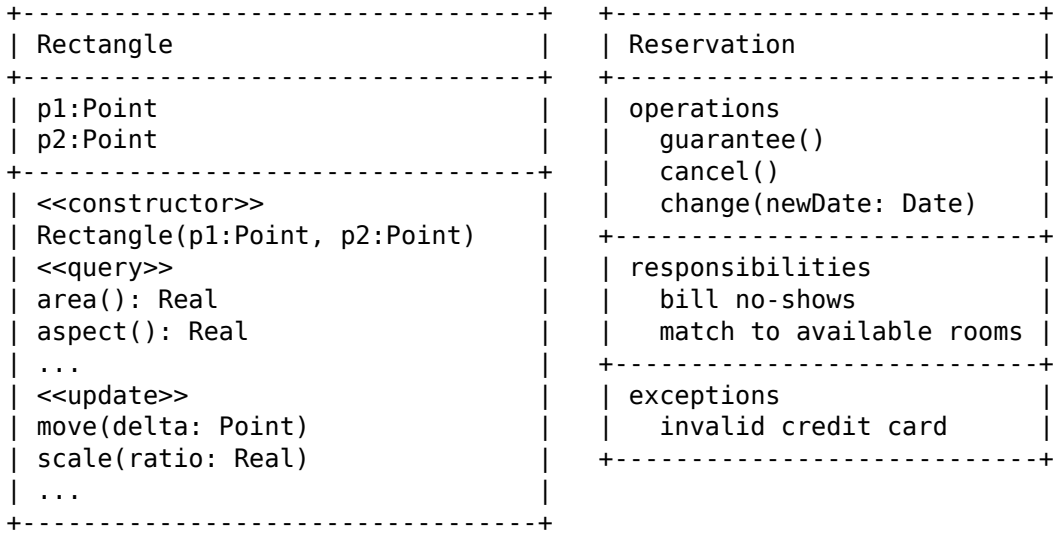
- İsimleri tanımlanmalıdır.

- Metotların görünürlüğü özelliklerdeki gibi tanımlanabilir.
- Değer döndüren işlemlerin dönüş tipi belirlenmelidir.
- Varsa parametreleri ve tipleri tanımlanmalıdır; istenirse ilk değer atamaları ve içeri/dışarı tanımları yapılabilir.
- Özel tanımlamalar `{..}` ile yapılabilir: `{query}` bazı özelliklerin bilgisini veren sorgu işlemidir; `{concurrency}`; `{abstract}`.
- **Sınıf kapsamı:** bu işlem sınıf kapsamındadır, tekil nesnelere ait değildir. Örnek: bu sınıftan kaç nesne türettik?

İşlem adları anlamlı olmalı, eğilimi ve sorumluluğu göstermelidir; ağaç örneğinde ağacın yığına eklenmesi, çekilmesi ve yok edilmesi için böyle adlar verilmeye çalışıldı. İçeri/dışarı tanımları, değerle çağırma (call by value) ve referansla çağırma (call by reference) gibi parametre geçirme biçimlerini göstermeye yarar.

10.5 Örnek sınıflar

Slayttaki iki örnek sınıf (ekrandan, 01:50:15):



Hoca Rectangle'ı şöyle açıklar: dikdörtgen iki noktayla, `p1` ve `p2` ile tanımlanır. Point de başka bir sınıftır ve bir yerde tanımlanmış olmalıdır. `<<constructor>>` stereotipiyle verilen kurucu, sınıfla aynı addadır ve Point tipinde iki parametre bekler. `<<query>>` metotları vardır: alanı hesaplayıp gerçek sayı döndüren `area()` ve dikdörtgenin oranını (4'e 3, 16'ya 9 gibi, ne kadar basık olduğunu) döndüren `aspect()`. `<<update>>` metotları özellikleri değiştirir: `move(delta)` dikdörtgeni delta kadar kaydırır, `scale(ratio)` verilen oranla ölçekler (ör. `p2` noktasını değiştirir).

Reservation sınıfının özellik bölmesi yoktur. Hoca rezervasyonun garanti etme, iptal ve yeni bir tarihle değiştirme işlemlerine sahip olmasını istediğini söyler. Sorumluluklar bölmesinde gelmeyen müşterilerin (no-show) faturalandırılması ve rezervasyonun yalnız boş odalara yapılması, dolu odalara yapılmaması vardır; bunlar bir şekilde garanti edilmelidir. Aykırı durumlar bölmesinde geçersiz kredi kartı durumunda ne yapılacağı yer alır. Bu sınıfta dört bölme vardır (ad, işlemler, sorumluluklar, aykırı durumlar); özellikler de eklenseydi beş olurdu. Hoca, o andaki ayrıntı ihtiyacına göre bir sınıfın yalnız adıyla, ad ve özellikleriyle ya da bütün bölmeleriyle gösterilebileceğini söyler.

10.6 İleri sınıf öğeleri

- **Arayüzler:** Bir tip tanımlar; sisteme ne sunulduğunu ve sistemden ne beklendiğini belirtir. `implements` ile hangi sınıfların bu arayüz şablonunu karşılayacağı, hangilerine uyacağı garanti edilir.
- **Parametrik sınıflar:** Java'da generic, C++'ta template olarak gerçeklenen yapılardır. Jenerik bir koleksiyondaki öğelerin türünü bir parametreyle tanımlamayı sağlar.
- **İç içe sınıflar** (nested class): sınıf içinde sınıf.
- **Kompozit nesneler:** içinde başka nesneler barındıran nesneler.

11. İleri ilişkiler: ilişki sınıfları, birleşme, bütünleşme, niteleyiciler

11.1 İlişki, genelleştirme ve bağımlılık örnekleri

- **İlişki:** İnsanlar araçları kullanır (insan ile araç arasında association).
- **Genelleştirme:** Araba bir araçtır.
- **Bağımlılık:** Araçların uyması gereken emisyon kanunları vardır. Emisyon kanunları değişirse arabaların buna uyarlanması gerekebilir. Görünmese de burada gizli bir bağımlılık vardır ve sınıf modelinde bir ilişki olarak mutlaka gösterilmelidir.

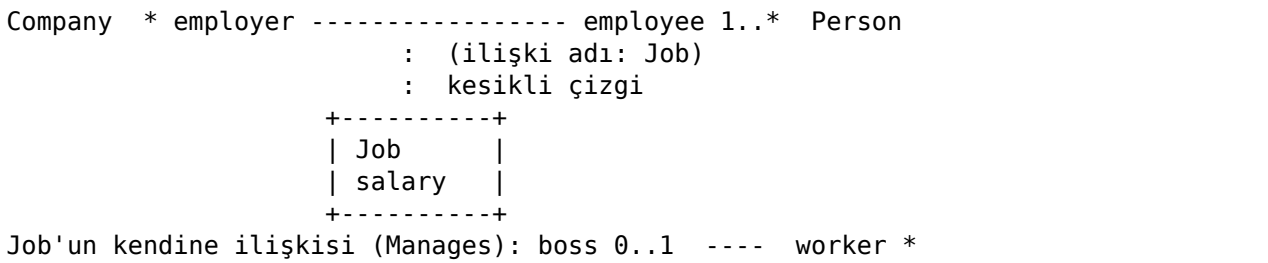
Slayttaki bir başka örnekte bir bütünleşme (composition) ve bir birleşme (aggregation) vardır; ilişkilerin yönü ve okuma yönü de gösterilebilir (ör. “Contains”). Bir poligon en az 3 ya da daha fazla noktadan oluşur; noktaların sıralı olması beklenir ve bu ilişkide noktalar `vertex` rolündedir, yani poligonun içindeki noktalara vertex olarak erişilir. Poligon ile grafik demeti (graphics bundle) arasında ise bütünleşme vardır; hocanın deyişiyle “mütemmim cüz”, ayrılmaz bir bütün.

İleri seviyede ilişkilerde yön, rol, okuma adı, çoğullama ve varsa kısıtlar gibi pek çok ayrıntı gösterilebilir.

11.2 İlişki sınıfları

Slayttaki tanım (ekrandan, 01:55:40):

- Sınıf nitelikleri taşıyan ilişkileri ayrı bir sınıf olarak modelleyebiliriz.
- “Kişi çalışır şirkette” ilişkisinde maaş özelliği tanımlayabiliriz. Maaş kişinin değil, çalışma ilişkisinin özelliğidir. Maaş şirketin özelliği de olamaz, şirketin pek çok çalışanı vardır. Kişinin birden fazla görevi için ayrı maaşları olabilir.
- İlişki sınıfları, ilişki çizgisine dayanan kesikli çizgilerle gösterilir.
- Burada İş sınıfının kendine ilişkisi vardır; buna özyinelemeli ilişki denir. Rol tanımlarını iki uçta da yapmak faydalıdır: patron yönetir işçiyi.



Hoca örneği şöyle anlatır: Person ve Company sınıfları employer (işveren) ve employee (çalışan) rolleriyle birbirini tanır ve aralarında Job ilişkisi vardır. Bu ilişkinin kendi özellikleri ve davranışları vardır; en basiti maaştır. Bir kişi bir şirkette belirli bir iş için belirli bir maaş alır. Çoğullama yıldız olduğu için birden fazla şirkette, ya da aynı şirkette başka bir pozisyonda çalışıp başka bir maaş alabilir. Özellik farklı değerler aldığına göre bunlar farklı nesneler olarak modellenmelidir. Bu yüzden ilişki çizgisine kesikli bir çizgiyle bir sınıf bağlanır: ilişki sınıfı. Job içinde kişinin patron mu (boss) işçi mi (worker) olacağı da modellenebilir.

> **Hocanın vurgusu:** Mümkünse ilişkileri yönleriyle birlikte çizin ve rol tanımlarını iki uçta da verin. İlişkinin kendi özellik ve davranışları varsa onu ayrı bir ilişki sınıfı olarak modelleyin.

11.3 Birleşme ve bütünleşme

Hoca iki kavramı yeniden ve ayrıntılı ele alır:

- **Birleşme (aggregation):** Bir sınıfın örneği, diğer sınıfın pek çok örneğinden yapılmıştır. İçi boş eşkenar dörtgenle gösterilir. İlişkinin anlamı, örneğin yaşam ömürleri hakkında bilgi vermez.
- **Bütünleşme (composition):** Parçanın yaşam süresi bütün tarafından yönetilir. Bütünleşme genellikle geçişkendir: bir bütünleşmenin içinde başka bir bütünleşme yer alabilir.

Hocanın ev örneği: Ev odalardan oluşur. Ev yıkılırsa odalar da yıkılır; odayı yıkmadan evi yıkmak olmaz. Bu yüzden ev ile oda arasında bütünleşme vardır: ev yok edildiğinde odaların yaşam ömrü de biter. Oda ile masa ise farklıdır. Masa odanın içindedir, ama ev yıkılırken masa dışarı alınıp başka yerde kullanılabilir. Yaşam ömürleri birbirine bağlı değildir; bu bir birleşmedir. Ev yıkılır, odalar yıkılır, masa yaşamaya devam eder.

11.4 Hangi ilişki?

Hoca slayttaki çiftler için ilişki türünü sorar (ekrandan, 02:00:00). Bu sorular "Örnek Sorular" bölümünde çözümleriyle verilmiştir. Hocanın açıklamaları:

- Dersler ve öğrenciler birleşmedir: öğrencileri yok etmeden ders kapatılabilir.
- Gelin ve damat basit bir ilişkidir: birinin içinde diğeri yoktur, bir kapsama durumu yoktur ve bağımsız yaşam ömürleri vardır.
- Banka hesabı ve mudi birleşmedir: mudinin banka hesapları vardır; mudi olmaktan çıkılsa da bir hesap uzun süre kapanmadan durabilir. Hoca bunun düz bir ilişki olarak da modellenebileceğini, yüzde yüz kesin olmadığını ekler.
- Yazı tipleri ve gölgeleri doğrudan bütünleşmedir: yazı tipi yok edilince bütün süslemeleri de onunla birlikte yok olur.

  Ek bilgi (derste anlatılmadı): "Mudi" bankacılıkta mevduat sahibi, yani bankaya para yatıran kişi anlamına gelir. Altyazıda bu kelime "Moody" olarak geçiyor; slayttaki yazım "Mudi"dir.

11.5 Niteleyiciler (qualifiers)

Slayttaki tanım (ekrandan, 02:01:20):

- Sınıf dörtgenlerinin kenarında yer alan küçük dörtgenlerdir.
- Sınıfın belirli bir özelliğinin adını taşır. Bu özellik sınıfın nesnelerine erişim sağlar; veritabanındaki birincil anahtar gibi düşünülebilir.
- Örnek: insanlar bankaya hesap numaraları ile erişir.
- Örnek: satranç tahtasında kareler koordinatla (satır-sütun) tanımlanır. Burada bütünleşme de vardır.

```
Bank [account#] *------0..1 Person

Chessboard [rank: Integer, file: Integer] 1 <*>------ 1 Square
                                        (bütünleşme, içi dolu eşkenar dörtgen)
```

Hoca satranç örneğini açar: tahtadaki siyah ve beyaz kareler tahtadan çıkarılıp alınamaz; tahta kırılıp atılırsa onlar da yok olur, bu yüzden bütünleşme vardır. Ama her karenin bir satırı ve sütunu, yani A5 ya da F8 gibi bir koordinatı bilinmelidir ve tahtadaki konum bu koordinatlarla belirlenir. Böyle özel erişim noktaları varsa niteleyicilerle ifade edilir. Banka örneğinde de kişi bankaya hesap numarası (ya da müşteri numarası) üzerinden erişir.

Niteleyiciler bağlantılarda (link) da kullanılabilir. Slayttaki nesne örneğinde bir kulüp ve onunla ilişkide üç kişi vardır (Jill, Joe ve Chris). Hepsi kulübün üyesidir. Bir kişi ayrıca Treasurer (sayman) bağlantısıyla bağlıysa officer rolündedir; Chris de üye olmanın yanında President bağlantısıyla officer bağlantısına sahiptir. Bir nesne başka bir nesneyle normal bir ilişki içinde olabilir, ama başka bir rol üzerinden ikinci bir erişim biçimi de modellenebilir.

12. Genelleme ve kısıtları

12.1 Genelleme gösterimi ve kısıtlar

Genelleştirmede bir üçgen kullanılır. Üçgen tarafındaki sınıf üst sınıf (ana sınıf), diğer taraftakiler alt sınıflardır (çocuk sınıflar). Alt sınıfın örnekleri aynı zamanda üst sınıfın da örneğidir.

> **Hocanın vurgusu:** Genellemeyi kalıtımla birebir aynı kavram gibi düşünmeyin; burada modelleme yapılıyor. UML'de çoklu üst sınıf ya da çoklu alt sınıf tanımlanabilir, alt sınıflar discriminator tanımlarıyla gruplanabilir.

Slayttaki kurallar (ekrandan, 02:04:00):

- Bir üst sınıftan türeyen alt alt sınıflar birden fazla alt sınıfa aitse `{overlapping}` kullanılır.
- Sadece tek bir üst sınıfa ait olursa alt alt sınıflar `{disjoint}`dir.
- Çocuk sınıfların durumu, kendisinden türeyen tüm nesneleri kapsıyorsa `{complete}`, kapsamıyorsa `{incomplete}`. Örneğin hibrit bir aracınız var ve çocukların hiçbirine uymuyor.

Slayttaki şekil:

```
Vehicle
  power ayrımı : WindPoweredVehicle, MotorPoweredVehicle   {overlapping}
  venue ayrımı : LandVehicle, WaterVehicle                 {overlapping}

Truck    --|> MotorPoweredVehicle, LandVehicle
Sailboat --|> WindPoweredVehicle, WaterVehicle
```

Vehicle sınıfının alt sınıfları iki ayrıma (discriminator) göre gruplanır: güç kaynağına göre (power) WindPoweredVehicle ve MotorPoweredVehicle; çalıştığı ortama göre (venue) LandVehicle ve WaterVehicle. Sailboat (yelkenli) hem rüzgârla çalışan hem de suda giden bir araçtır; Truck (kamyon) hem motorlu hem kara aracıdır. Alt alt sınıflar birden fazla alt sınıfa ait olduğu için ilişki overlapping'tir.

Hoca incomplete durumunu bir aracın hiçbir alt sınıfa uymamasıyla örnekler: ne rüzgârla ne motorla çalışan, ne kara ne su aracı sayılabilen hibrit bir araç (hoca hovercraft'ı örnek verir). Böyle bir araç varsa kısıt incomplete'tir.

12.2 Atlet ve kitap örnekleri

Slayttaki örnekler (ekrandan, 02:06:10) "Örnek Sorular" bölümünde çözümleriyle verilmiştir. Hocanın gerekçeleri:

- Atlet sınıfının alt sınıfları VoleybolOyuncusu, BasketbolOyuncusu ve FutbolOyuncusu'dur. Bir oyuncu hem voleybol hem futbol, ya da hem basketbol hem futbol oynayabilir; bu yüzden overlapping'tir. Bunları oynamayan atletler de vardır (ör. hentbol oyuncusu); bu yüzden incomplete'tir.
- Kitap sınıfının alt sınıfları KağıtCiltli, ÇizgiRoman (fasikül) ve BezCiltli'dir. Tekil bir kitap aynı anda hem kâğıt hem bez ciltli olamaz; ISBN numaraları da farklı olmak zorundadır. Bu yüzden disjoint'tir. Günümüzde e-kitap gibi başka türler olduğu için de incomplete'tir.

12.3 Omnivor mu, vejetaryen mi?

Hoca kafa karıştırıcı bir örnek verir: omnivorlar mı üst sınıftır, vejetaryenler mi? Omnivorlar hem et hem ot yer, vejetaryenler et yemez. Omnivor daha geniş bir kavram gibi görünür. Ancak hocaya göre vejetaryen üst sınıftır, çünkü alt sınıf üst sınıfın özelleşmiş hâlidir. Vejetaryenlerde daha az davranış vardır; omnivorlar bunun üzerine bir şeyler ekleyerek onu özelleştirir. Bu yüzden vejetaryen daha genel bir kavramdır.

12.4 UML diyagramlarının hiyerarşisi

Ders, ilk saatte anlatılan diyagramlara dönerek kapanır. UML diyagramları yapısal ve davranışsal diye ikiye ayrılır ve her birinin altında 7 diyagram vardır. Bu hiyerarşi de bir genelleştirme ilişkisidir: hepsi UML'de Diagram sınıfının altında yer alır. Böylece bu hafta bahsedilen 14 diyagramın nasıl dağıldığı tek bir genelleştirme ağacında görülür.

Örnek Sorular

Soru 1 (ekrandan, 02:00:00; derste sözlü, 01:59:00). Aşağıdaki çiftler arasındaki ilişki birleşme (aggregation), bütünleşme (composition) ya da düz ilişki (association) türlerinden hangisidir?

- a) Dersler ve öğrenciler
- b) Gelin ve damat
- c) Banka hesabı ve mudi
- d) Yazı tipi ve gölgesi (derste sözlü, 02:00:37)

Çözüm (derste verildi):

- a) Birleşme. Öğrencileri yok etmeden ders kapatılabilir.
- b) Düz ilişki. Biri diğerinin içinde değildir ve bağımsız yaşam ömürleri vardır.
- c) Birleşme. Mudinin banka hesapları olabilir; mudilikten çıkılsa da hesap kapanmadan durabilir. Hoca bunun düz ilişki olarak da modellenebileceğini, kesin olmadığını söyledi.
- d) Bütünleşme. Yazı tipi yok edilince gölge gibi süslemeleri de onunla birlikte yok olur.

Soru 2 (ekrandan, 02:06:10). Aşağıdaki genellemeler `{overlapping}` mi `{disjoint}` mi, `{complete}` mi `{incomplete}` mi?

- a) Atlet: VoleybolOyuncusu, BasketbolOyuncusu, FutbolOyuncusu
- b) Kitaplar: KağıtCiltli, ÇizgiRoman, BezCiltli

Çözüm (derste verildi):

- a) Overlapping ve incomplete. Bir oyuncu hem voleybol hem futbol oynayabilir; bunları oynamayan atletler de vardır (ör. hentbol oyuncusu).
- b) Disjoint ve incomplete. Bir kitap aynı anda hem kâğıt hem bez ciltli olamaz; e-kitap gibi listede olmayan türler vardır.

Soru 3 (derste sözlü, 02:06:44). Omnivorlar ile vejetaryenler arasındaki genellemede hangisi üst sınıftır?

> **Çözüm (derste verildi):** Hocaya göre vejetaryen üst sınıftır. Alt sınıf, üst sınıfın özelleşmiş hâlidir. Omnivor, vejetaryenin davranışına et yemeyi ekleyerek onu özelleştirir; vejetaryen daha az davranış içeren, daha genel kavramdır.

Soru 4 (derste sözlü, 01:34:57). Ağaç yapraklarını sayma örneğinde sayma sorumluluğu kime verilmelidir: ağacın kendisine mi, sayaca mı?

> **Çözüm (derste verildi):** Karar tasarımcıya aittir ve iki seçenek de bir çözümdür. Sayaç, adı gereği bir şeyleri sayması beklenen sınıftır ve artırma işlemi onun uzmanlık alanıdır. Ağaç kendini sayarsa kendi içinde gereksiz işler yapmaya başlar. Karar iyi uyum ve az bağımlılık ilkelerine göre verilmelidir. Hocanın modelinde artırma ve gösterme Counter'da, bölme ve atma Tree'de, koyma ve çekme Pile'dadır.

Soru 5 (ekrandan, 00:37:20). Slayttaki use case diyagramında kaç kullanım senaryosu yazılmalıdır? `<<uses>>` ve `<<extends>>` ilişkileri neyi ifade eder?

> **Çözüm (derste verildi):** Diyagramda 7 elips vardır (SetLimits, UpdateAccounts, AnalyzeRisk, PriceDeal, CaptureDeal, Valuation, LimitsExceeded), yani 7 senaryo yazılmalıdır. AnalyzeRisk ve PriceDeal, ortak bir alt senaryo olan Valuation'ı `<<uses>>` ile çağırır (alt program çağrısı gibi). LimitsExceeded, CaptureDeal'ı `<<extends>>` ile genişletir: CaptureDeal tek başına da, limit aşımı durumunda bu ek parçayla birlikte de çalışabilir.

Dersin Toparlaması

Dersin ilk saatinde UML 2'nin 14 diyagramı tanıtıldı. Yapısal diyagramlar (sınıf, nesne, kompozit yapı, bileşen, barındırma, paket, profil) sistemin her zaman var olan parçalarını; davranışsal diyagramlar (use case, aktivite, ardışıl, iletişim, genel etkileşim, zamanlama, durum) bu parçaların nasıl çalıştığını gösterir. Bir mimarinin statik yapısı için sınıf, bileşen, barındırma ve paket diyagramları, davranışı için çoğunlukla ardışıl diyagramlar yeterlidir. Diyagramla ifade edilemeyen kısıtlar OCL ile, UML'in kendisinin genişletilmesi ise profil diyagramlarıyla yapılır.

İkinci saatte nesneye yönelik analizin mantığı ve yöntemi gösterildi. Nesneler fonksiyonlardan daha kararlı olduğu için daha sürdürülebilir tasarımlar sağlar. Gereksinim metnindeki isimler sınıflara, sıfatlar özelliklere, eylem fiilleri işlemlere, durum fiilleri ilişkilere dönüştürülür. Ancak her adımda tasarımcının düşünerek karar vermesi gerekir ve tek doğru model yoktur. Ardından UML sınıf modelinin ayrıntıları (görünürlük, çoğullama, stereotipler, ilişki sınıfları, birleşme ve bütünleşme, niteleyiciler, genelleme kısıtları) örneklerle ele alındı. Hoca bir sonraki derste basit bir kütüphane örneği üzerinden tasarım çalışmalarına geçileceğini ve formal tanımlara bakılacağını, zaman kalırsa OCL'ye de girileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.